Quest 6: The Nullspiracy

Lord of the Wings · following ingredient records forwards and backwards

> This guide uses the official puzzle text. The story is summarised and the rules are restated in plain words. It explains what the quest asks and the logic behind my solutions, not the code itself. Every number in the examples and traces was checked by running my solutions on the sample data (plus small instrumented copies to print intermediate values).

THE QUESTION

Given "item is made from these ingredients" records, count what is reachable from `NULL` (Part I), what can reach `NULL` (Part II), and what can reach two different versions of `NULL` (Part III).

INPUT

One record per line, `ITEM:ING1,ING2,...`, with four-letter identifiers.

OUTPUT

A count of distinct identifiers.

WHAT CHANGES BETWEEN PARTS

Part I walks *down* from `NULL` to its ingredients. Part II walks *up*: which products contain `NULL` somewhere inside. Part III splits `NULL` into three versions and asks which products contain at least two of them.

The story

At a roadside inn the egg starts smoking near the bread, so the company suspects contaminated food. The innkeeper has records of what every product was made from, and a suspicious identifier, `NULL`, keeps turning up. The heroes first check what `NULL` is made of, then what is made *with* it, and finally learn that there are three look-alike versions of `NULL`, and it's mixing versions that is dangerous.

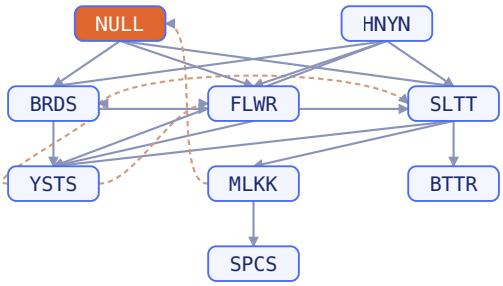
Understanding the question

The records form a graph

Each record `BRTS:FLWR,SLTT,MLKK` is a set of arrows: `BRTS → FLWR`, `BRTS → SLTT`, `BRTS → MLKK` ("is made from"). All the records together form a **directed graph**, which my solution stores as a lookup from each item to its ingredients. A few things the puzzle points out:

- An ingredient can be used by many items, so the same node can be reached along several paths. Count it **once**.
- The graph can have **cycles** (in the sample, `BRDS` uses `YSTS` and `YSTS` uses `BRDS`). A walk that doesn't remember where it has been would go round forever.
- An identifier that never appears on the left of a `:` is a basic ingredient with no arrows out of it.

"Used directly or indirectly" means "reachable by following arrows". The set of everything reachable from a node is its **transitive closure**. The Part I sample as a graph, laid out by distance from `NULL`:



solid arrows go down a level; dashed ones point back up (that's where the cycles are)

Forwards vs backwards

Part I follows the arrows forwards from `NULL` (what is it made of?). Part II asks the opposite: from which items can you follow the arrows and arrive at `NULL`? That's the same as following the arrows **backwards** from `NULL`.

Extra examples of my own (all run through my solution)

Part	Records	Answer	Why it's interesting
I	`NULL:AAAA` / `AAAA:BBBB` / `BBBB:NULL`	2	The chain loops back to `NULL`. `NULL` itself is not counted, and the loop doesn't run forever.
I	`NULL:AAAA,BBBB` / `AAAA:CCCC` / `BBBB:CCCC`	3	A "diamond": `CCCC` is reached twice but counted once.
I	`AAAA:BBBB` (no record for `NULL`)	0	Nothing to expand.
II	`NULL:XXXX` / `AAAA:BBBB` / `BBBB:AAAA` / `CCCC:NULL`	1	Only `CCCC`. The `AAAA`/`BBBB` cycle never reaches `NULL`.
II	`NULL:AAAA` / `AAAA:NULL` / `BBBB:AAAA`	2	`AAAA` and `BBBB`. `NULL` is in a cycle with itself but isn't counted as a product.
III	`PIEE:NULLv1,NULLv1`	0	The same version twice is still only one version.
III	`JAMM:NULLv1` / `TART:JAMM,NULLv2` / `CAKE:TART`	2	`TART` (v2 directly, v1 through `JAMM`) and `CAKE` (both through `TART`).

PART I What is NULL made of?

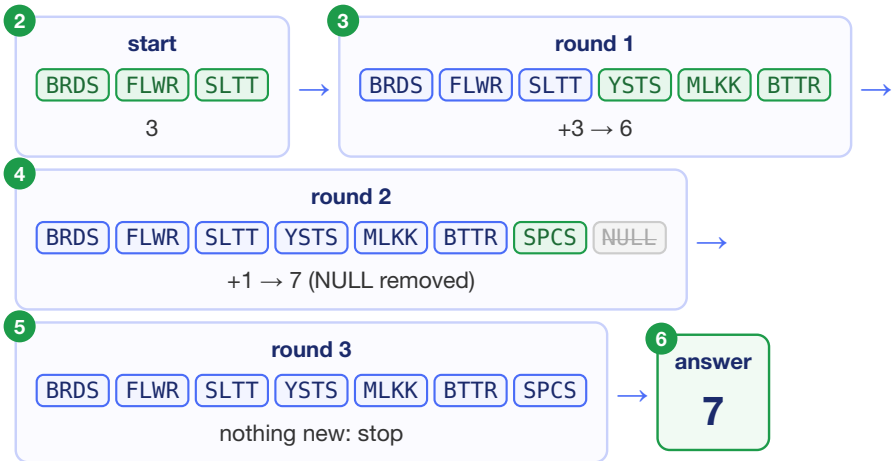
Starting from `NULL`, collect every ingredient you can reach by following the records, counting each one once and leaving out `NULL` itself. How many are there?

Step-by-step solution

1. Read the records into a table from item to its list of direct ingredients.
2. Start with `NULL`'s list.
3. In each round, append the ingredients of every entry in the list.
4. Remove `NULL` and duplicates.
5. If the list didn't get longer, stop. Otherwise do another round.
6. Print the length of the list.

How my solution works

Keep one list, starting with `NULL`'s ingredients. Each **round** appends the ingredients of everything already in the list, then removes `NULL` and duplicates. Stop when a round adds nothing new:



Dots match the step numbers above. Blue = already in the list, green = new this round, struck out = removed.

> **Why the cycle doesn't hang:** `BRDS` → `YSTS` → `BRDS` is a cycle, but each round just re-adds things that are already in the list, the clean-up throws away the duplicates, and the length stops growing. The search ends when a whole round finds nothing new: a **fixed point**.

> **Cost:** each round re-expands *every* entry, not just the new ones. A **breadth-first search** with a "seen" set expands each item once; for an input this size, my version is still instant.

PART II What uses NULL?

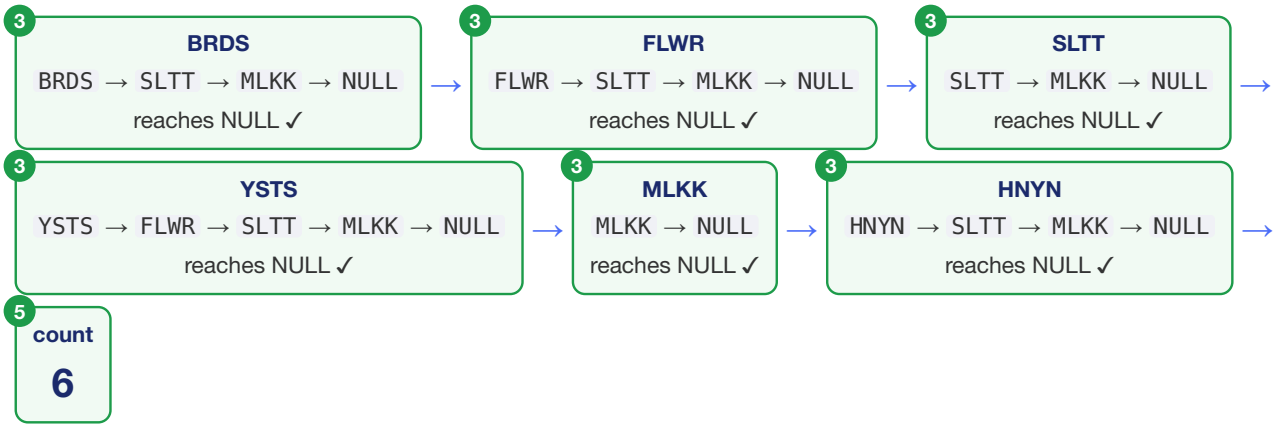
Now go the other way: count the products (other than `NULL`) that contain `NULL` directly or indirectly.

Step-by-step solution

1. Read the records as in Part I.
2. Turn Part I's search into a general tool: given a starting list, return everything reachable from it (keep `NULL` in the result).
3. For every product except `NULL`, compute the full closure of a **copy** of its list, so the stored records stay unchanged.
4. If `NULL` is in it, count the product.
5. Print the count.

How my solution works

Part I's search becomes a tool: "everything reachable from this list" (keeping `NULL` in the result, never expanding it). Run it once per product, from a **copy** of its record, and count the products that reach `NULL`:



Dots match the step numbers above. Each panel shows one route the search finds. The puzzle's intended idea is the reverse: one walk *up* from `NULL` (NULL ← MLKK ← SLTT ← BRDS, FLWR, HNYN ← YSTS) gives the same 6.

> **Why start from a copy:** the search appends to the list it's given. An earlier version passed the stored record itself, so records quietly grew (BRDS 3 → 10 entries, HNYN up to 47; about 7× on the real input). It still got the right answer, since the extra entries were already reachable, but only by luck. With the copy, the records stay unchanged.

> **Cost:** one full search per product. A single walk over the **reversed graph** from `NULL` would touch each record once. On the real input, my version takes about a second.

PART III Two different versions of NULL

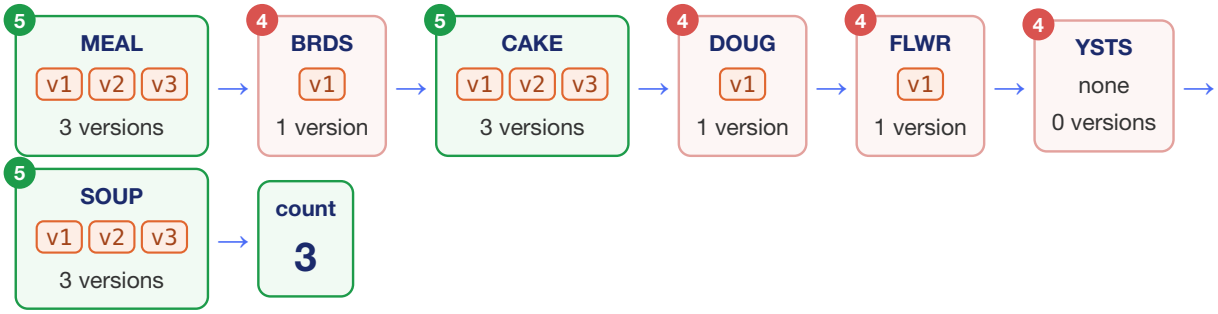
`NULL` now comes in three versions: `NULLv1`, `NULLv2` and `NULLv3`. Count the products that use **at least two different versions**, directly or indirectly.

Step-by-step solution

1. Read the records as in Part I.
2. Use the same search, but don't expand the three `NULL` versions.
3. For every product that isn't a version, compute the full closure of a copy of its list.
4. Count how many of `NULLv1`, `NULLv2`, `NULLv3` are in it.
5. If it's at least 2, count the product. Print the count.

How my solution works

The same search per product (not expanding the three versions, which aren't products), then count how many **different** versions turn up. Two or more → counted:



Dots match the step numbers above. MEAL and SOUP get all three through CAKE (v2, v3 directly, v1 via DOUG). A record like `PIEE:NULLv1,NULLv1` counts as 1 version, not 2.

> **Starting from a copy matters here too:** without it the records grew about 7× on the real input (5.1 s instead of about 4 s), with the same answer.

Summary

Part	Question	Direction	How my solution does it	Sample
I	ingredients of `NULL`	forwards from `NULL`	grow `NULL`'s list level by level until its length stops changing	7
II	products that use `NULL`	backwards to `NULL`	forward closure of every product (from a copy of its record); check for `NULL`	6
III	products using ≥ 2 versions	backwards to each version	forward closure of every product (from a copy of its record); count distinct versions	3

Key ideas: the records are a directed graph with cycles; "directly or indirectly" means the **transitive closure**; removing duplicates (or keeping a "seen" set) is what makes cycles and repeated paths safe; and asking "who uses X" is the same as searching forwards in the **reversed graph**.